Project:Highly Available Web Application with Auto Scaling on AWS

1.Project Overview

Objective

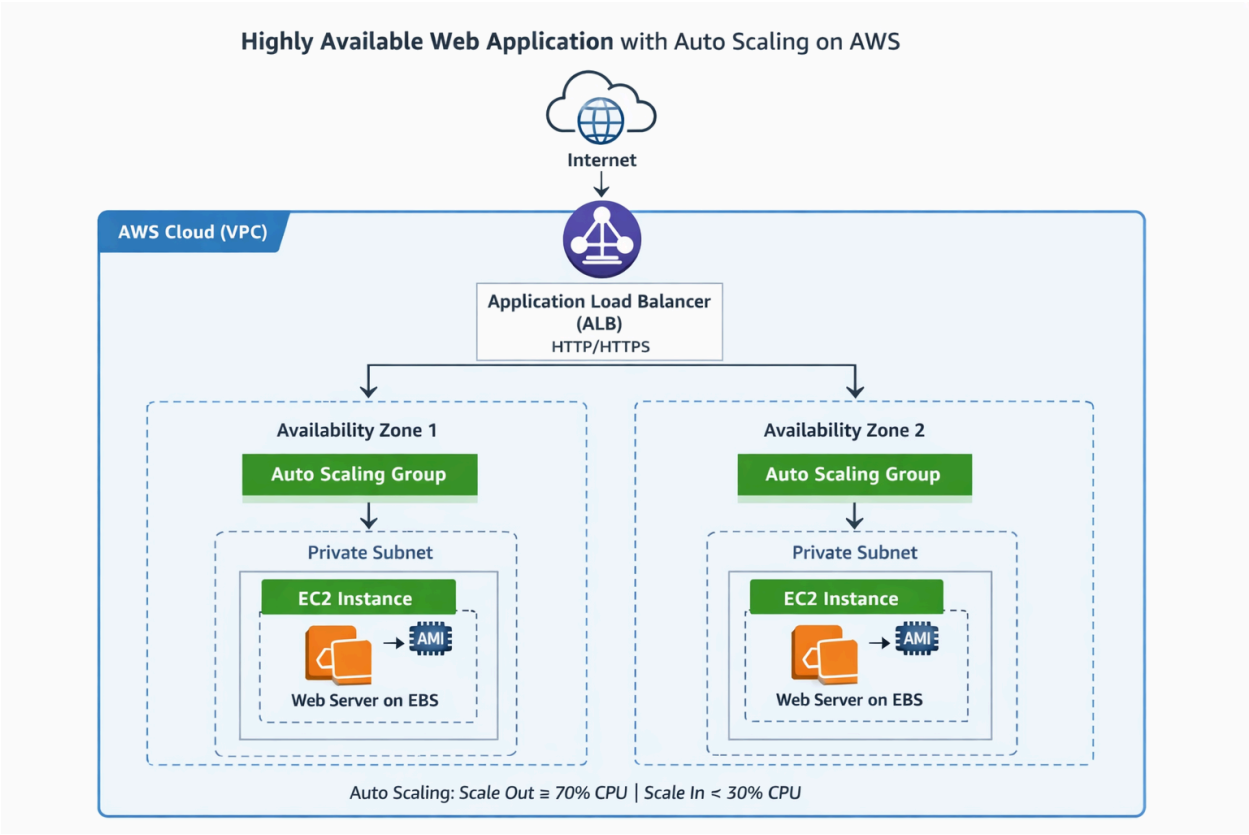
In this project we are going to design and deploy a **highly available**, **self-healing**, and **auto-scaling web application infrastructure** using different AWS services like:

- Amazon EC2
- Application Load Balancer
- Auto Scaling Group

The infrastructure must:

- Use minimum 2 Availability Zones
- Automatically scale based on CPU usage
- Replace unhealthy instances
- Prevent direct access to EC2
- Be production-ready

2.Architecture Diagram



3.Architecture Components

These are the components that can be used for the given requirement:

1. VPC

- Custom VPC
- 2 Public Subnets (for ALB)
- 2 Private Subnets (for EC2)

2. Application Load Balancer

- Internet-facing
- Deployed in 2 public subnets
- Listener: HTTP (Port 80)
- Target Group attached to EC2 instances

3. Auto Scaling Group

- Minimum capacity: 2
- Desired capacity: 2
- Maximum capacity: 5
- Distributed across 2 AZs

4. Launch Template

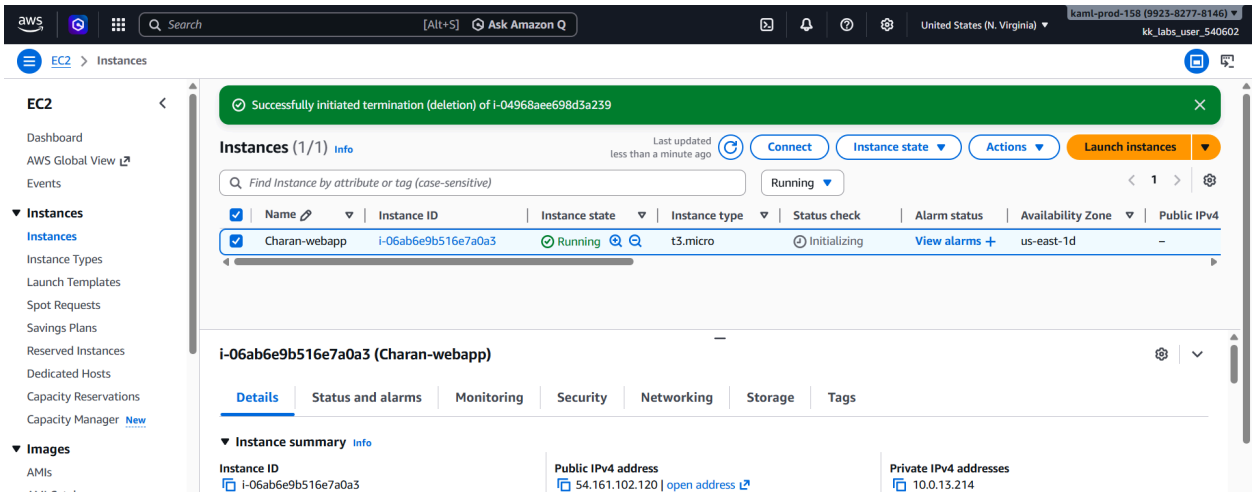
- Custom AMI
- Instance type: t2.micro / t3.micro
- Security group attached
- EBS storage configured

4.Step-by-Step Implementation

Step 1: Create and Configure Base EC2 Instance

1. Launch EC2

- Choose Amazon Linux / Ubuntu
- Instance type: t3.micro
- Attach Security Group:
    - Allow HTTP (80)
    - Allow SSH (22) from your IP only



---

2. Install Web Server

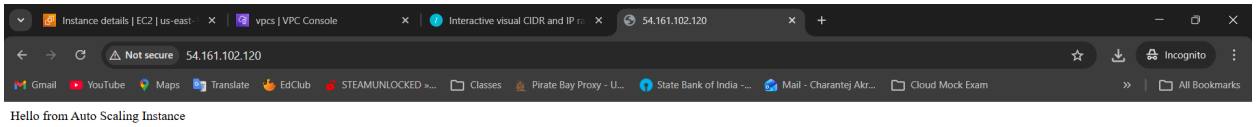
For Ubuntu:

```
sudo apt update
sudo apt install apache2 -y
sudo systemctl start apache2
```

Create test page:

```
echo "Hello from Auto Scaling Instance" | sudo tee /var/www/html/index.html
```

Verify in browser.

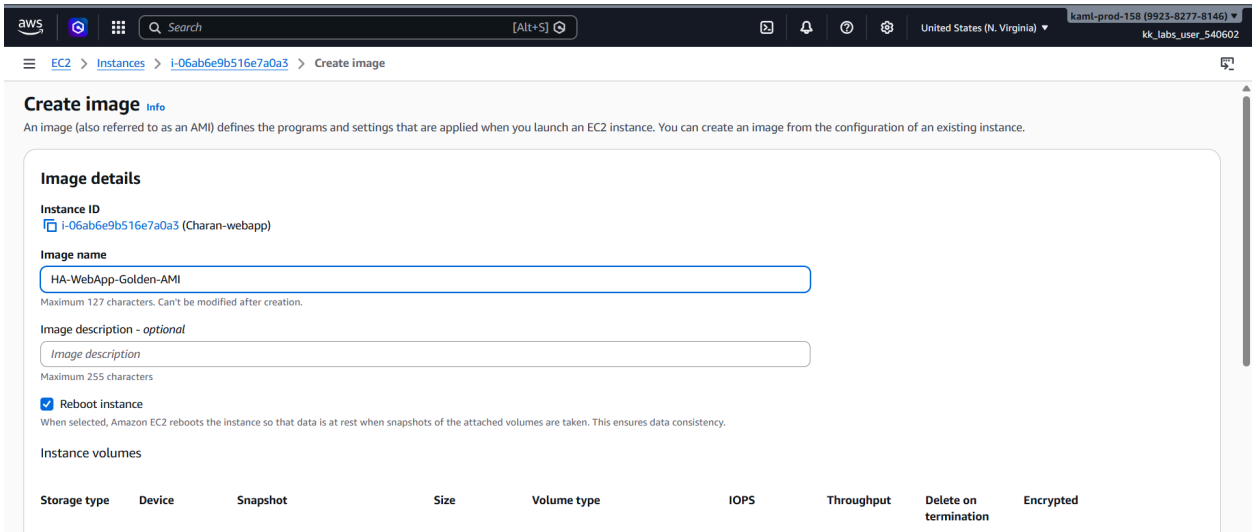


Step 2: Create Custom AMI (Golden AMI)

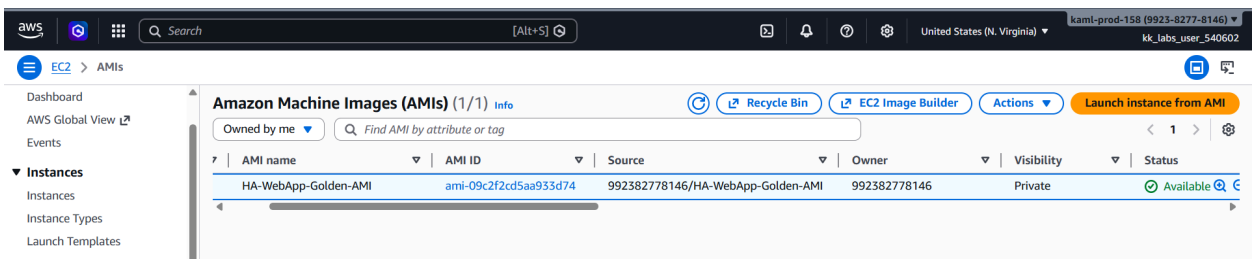
1. Go to EC2 → Instances
2. Select configured instance
3. Click **Actions** → **Image** → **Create Image**

Name:

HA-WebApp-Golden-AMI



Wait until AMI status becomes **Available**.

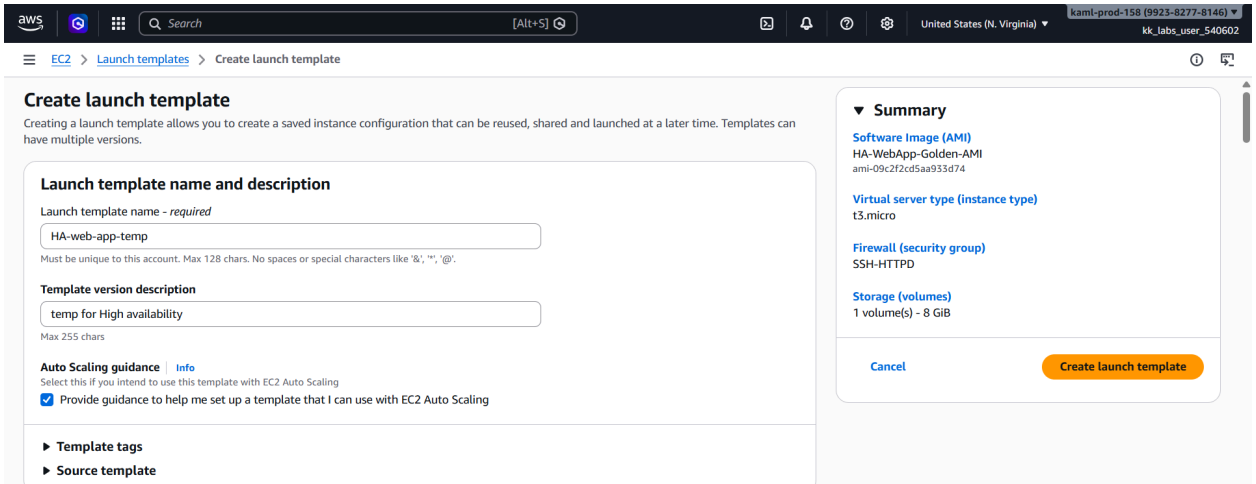


Step 3: Create Launch Template

1. Go to EC2 → Launch Templates
2. Click Create Launch Template

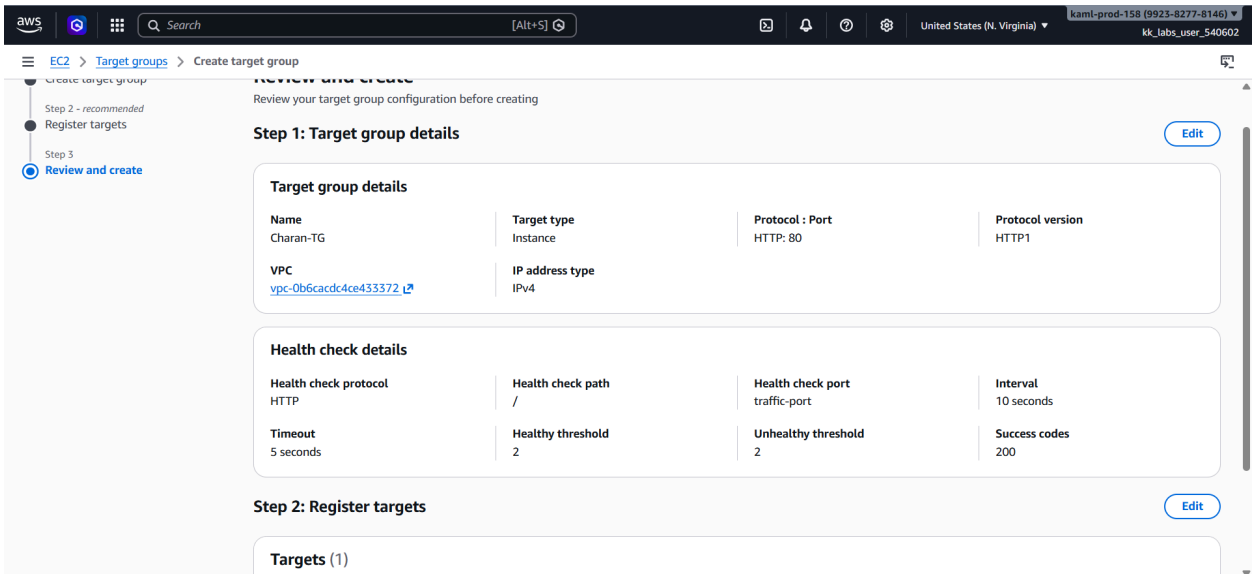
Configure:

- AMI → Select custom AMI
- Instance type → t3.micro
- Key pair → Select your key
- Security Group → EC2 Security Group
- Storage → Default EBS



Step 4: Create Target Group

1. Go to EC2 → Target Groups
2. Choose Target type → Instances
3. Protocol → HTTP
4. Port → 80
5. Health Check Path → /
6. Healthy threshold → 2
7. Unhealthy threshold → 2



Step 5: Create Application Load Balancer

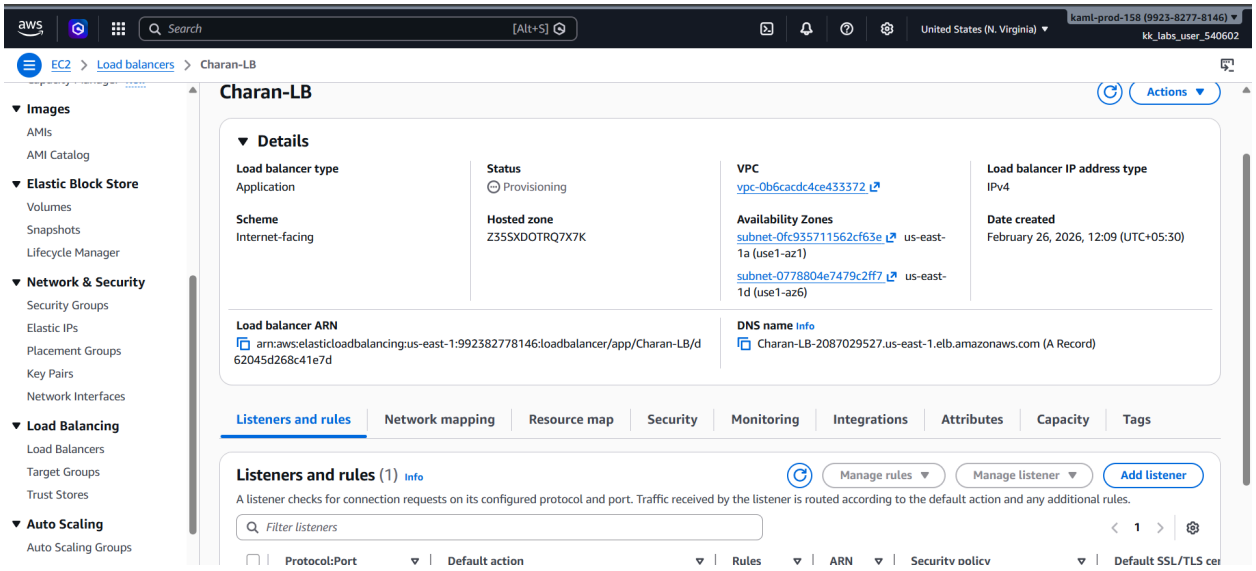
1. Go to EC2 → Load Balancers
2. Choose **Application Load Balancer**

Configure:

- Scheme → Internet-facing
- IP type → IPv4
- Subnets → Select 2 public subnets (different AZs)
- Security Group:
    - Allow HTTP (80) from 0.0.0.0/0

Listener:

- HTTP (80)
- Forward to Target Group



Step 6: Configure Security Groups (Layered Security)

ALB Security Group

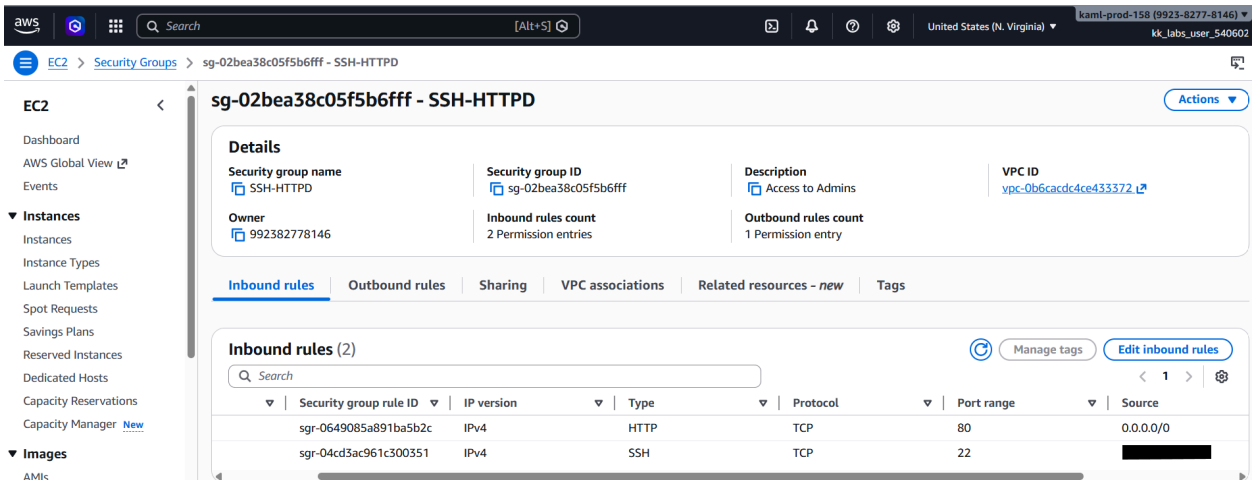
Allow:

- HTTP 80 → 0.0.0.0/0

EC2 Security Group

Allow:

- HTTP 80 → Source = ALB Security Group
- SSH 22 → Your IP only



Step 7: Create Auto Scaling Group

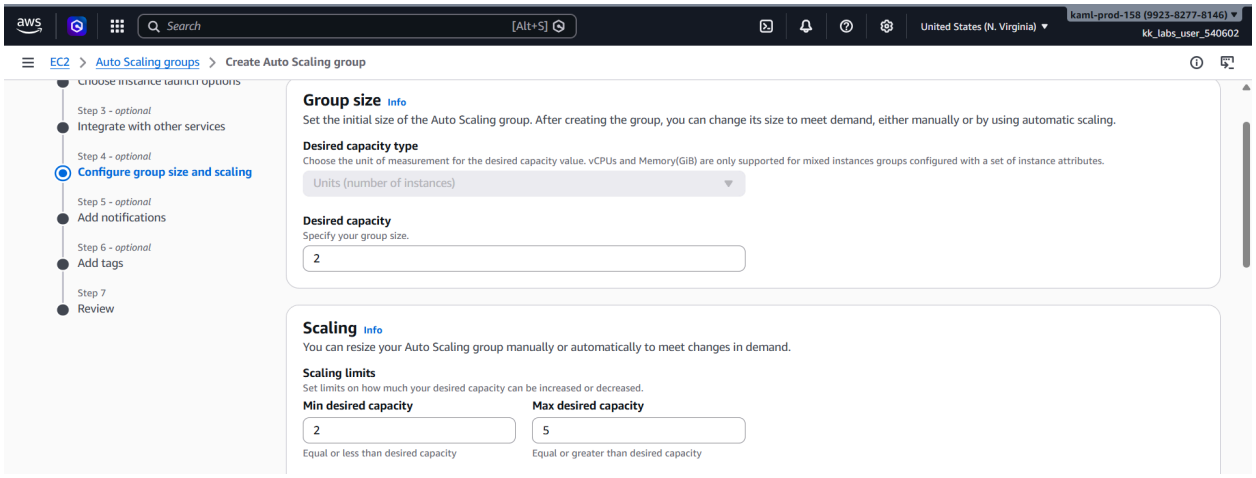
1. Go to EC2 → Auto Scaling Groups
2. Click Create

Configuration:

- Launch Template → Select created template
- Subnets → Select 2 private subnets (different AZs)
- Attach to existing Target Group

Capacity:

- Minimum: 2
- Desired: 2
- Maximum: 5



Step 8: Configure Scaling Policies

Create Simple Scaling Policy using Cloudwatch Metrics:

-In CloudWatch create two metrics one for Scale Out and Another one for Scale In.

- Then after creation of Auto Scaling group go to the Automatic scaling section under your ASG there create a Dynamic scaling policy under that create 2 policy one for the Scale Out and another for Scale In and Attach the CloudWatch Metrics for the Corresponding Policies to the respective metrics.

Metric:

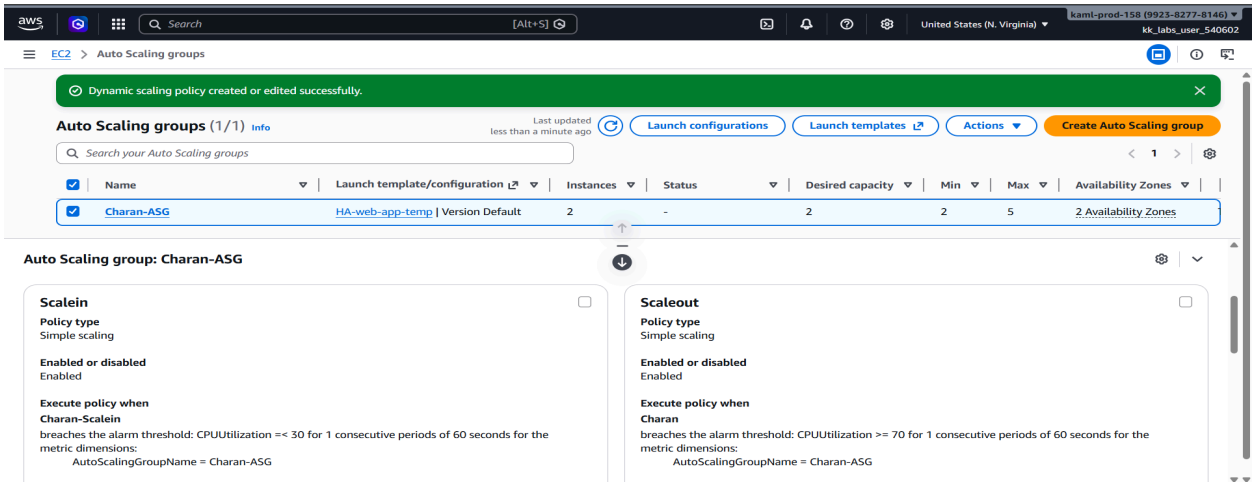
- Average CPU Utilization

Scale Out:

- When CPU ≥ 70%

Scale In:

- When CPU ≤ 30%



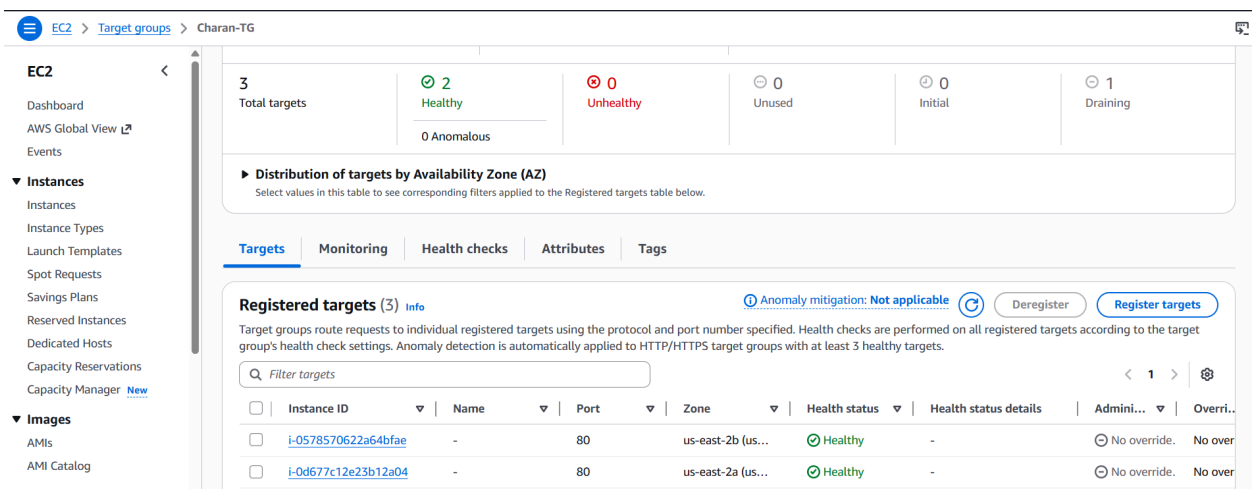
Step 9: Verify Health Checks

Go to:

- Target Groups → Targets

Ensure:

- Both instances show **Healthy**



Step 10: Demonstrate Scale-Out Event

SSH into one instance using Bastion Host and NAt gateway route to the Private Route Table:

Install stress tool:

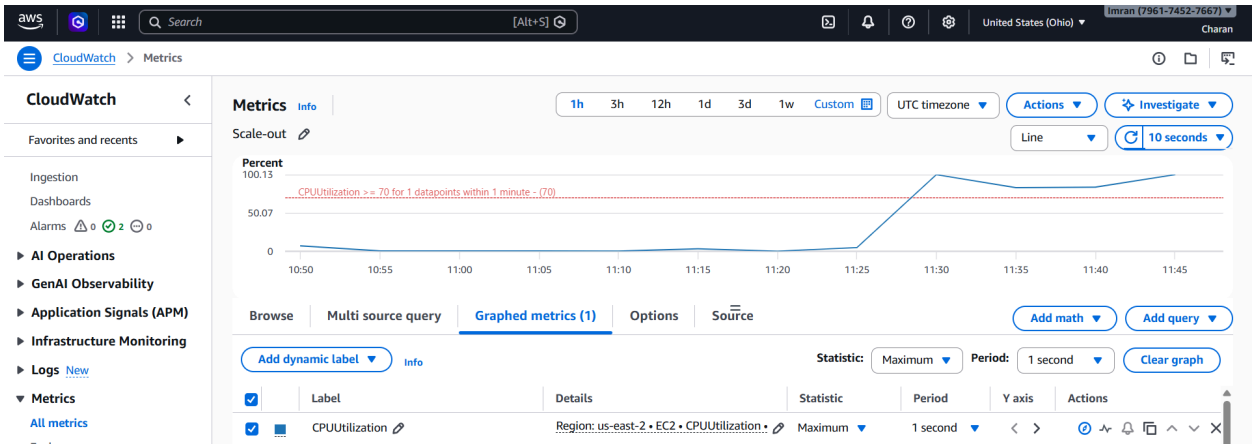
sudo apt install stress-ng -y

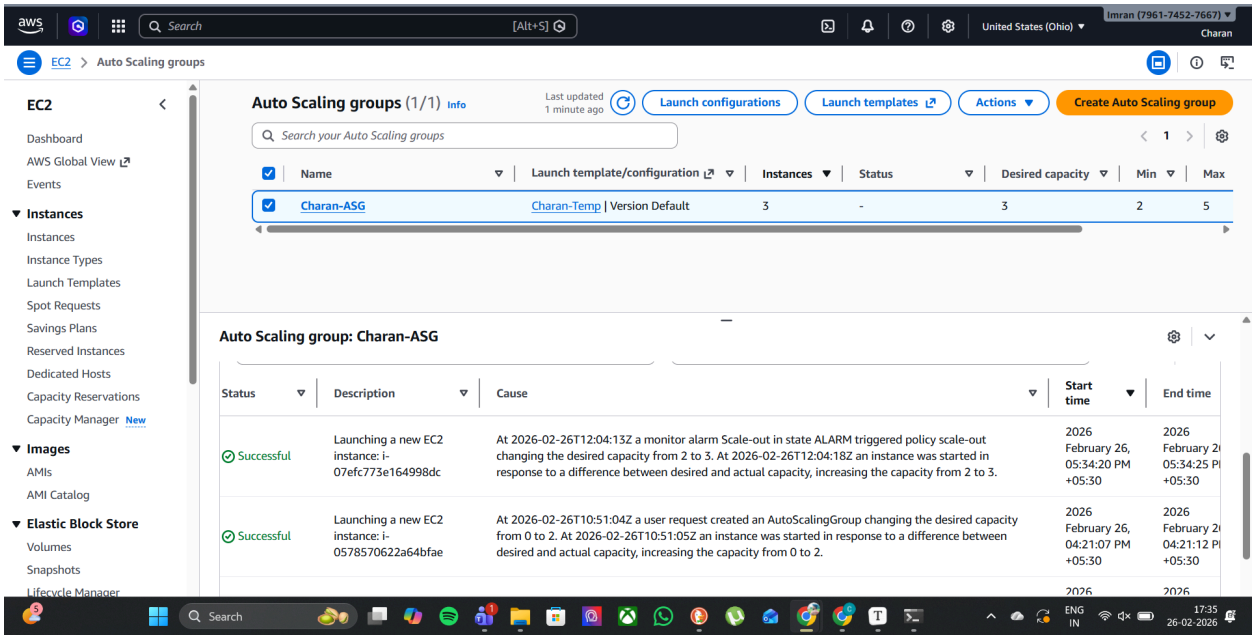
Run:

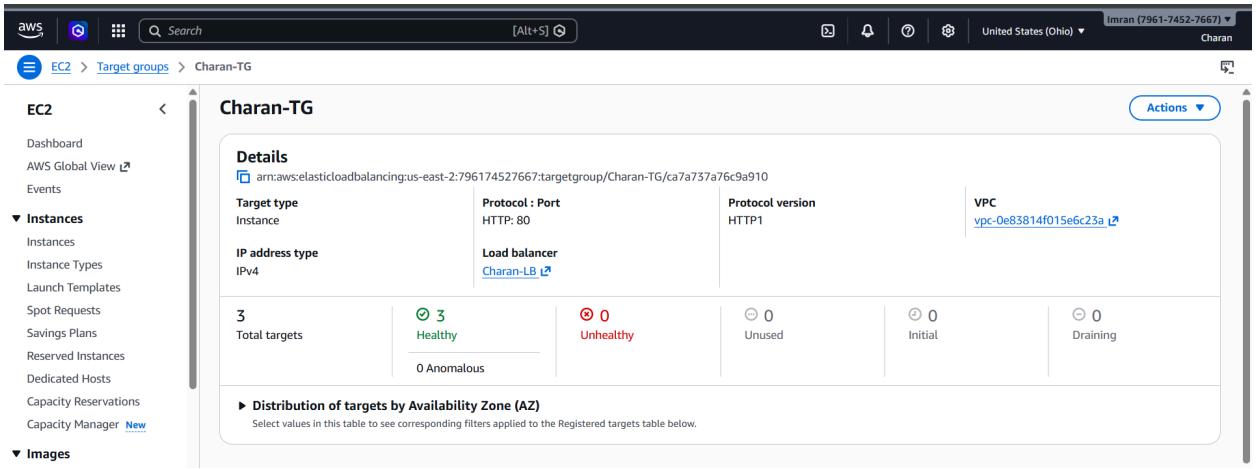
stress-ng --cpu 2 --cpu-load 70 --timeout 300

Monitor:

- CPU > 70%
- ASG launches new instance
- New instance registers in target group



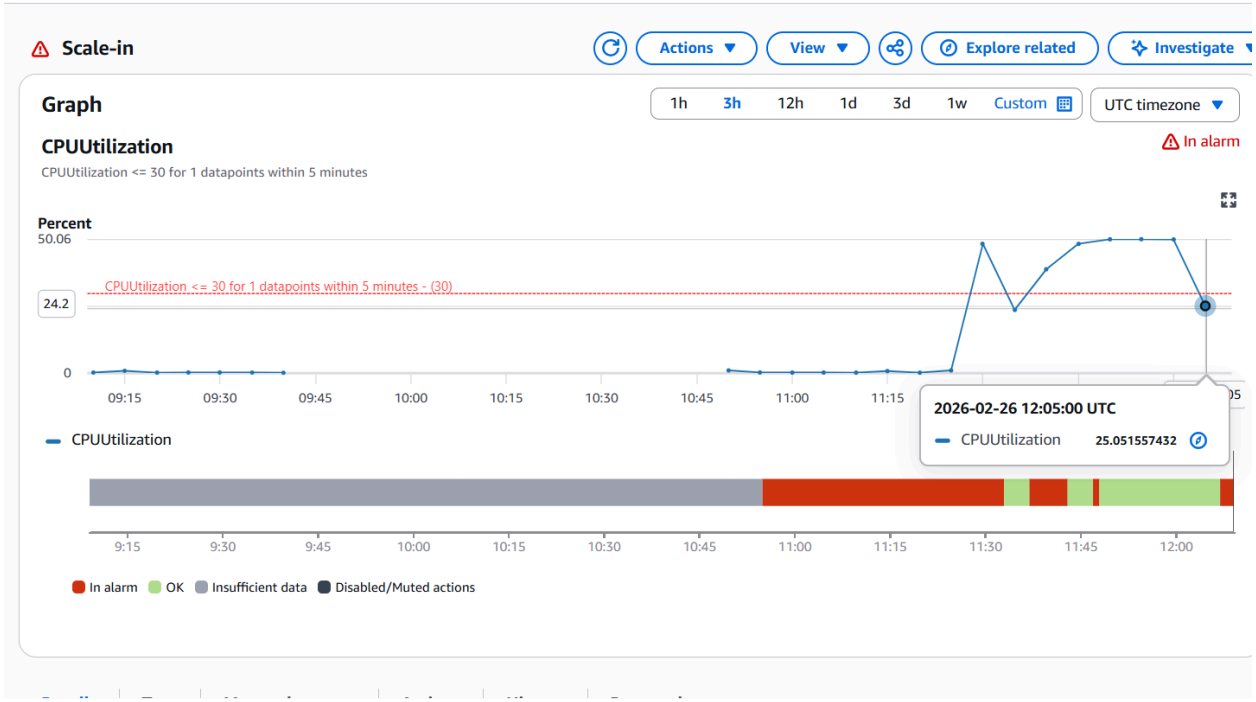


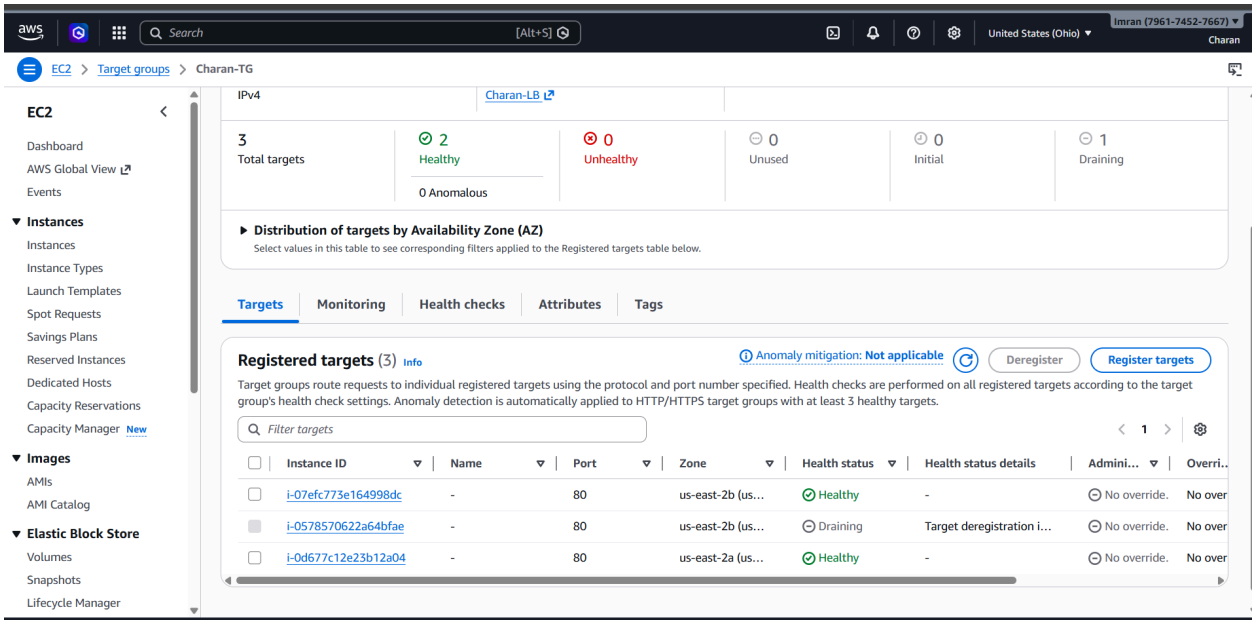


Step 11: Demonstrate Scale-In Event

After CPU drops:

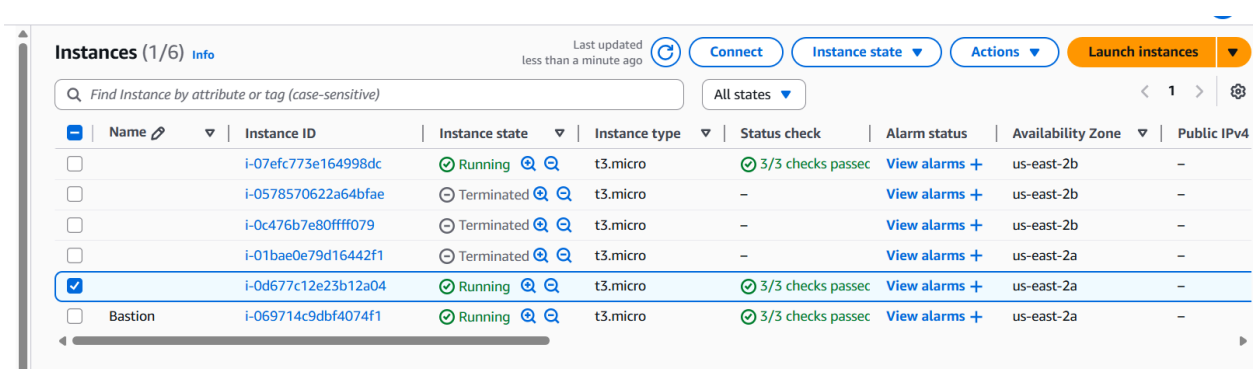
- ASG reduces instance count
- Instances terminate gracefully





5.Testing High Availability

1. Manually terminate one instance.
2. Observe:
    - ASG launches new instance.
    - No downtime from ALB.



6.Why ALB Instead of NLB?

- ALB works at Layer 7 (HTTP/HTTPS).

- Supports path-based routing.
- Supports host-based routing.
- Supports SSL termination.

NLB works at Layer 4 (TCP/UDP) and is better for non-HTTP workloads.

7.Why Launch Template Instead of Launch Configuration?

- Because it supports versioning.
- It also supports the latest EC2 features.
- Required for mixed instance policies.

8.Production Concepts Demonstrated

- Multi-AZ Design
- Self-Healing Infrastructure
- Horizontal Scaling
- Health Check Configuration
- Secure Security Group Layering
- Load Balancing at Layer 7

9.Conclusion

This project demonstrates a production-ready, highly available infrastructure capable of:

- Handling unpredictable traffic spikes
- Automatically scaling based on demand
- Recovering from instance failure
- Providing continuous service without downtime

The architecture follows cloud best practices for scalability, security, and reliability.